

AWS Python Deployment

Session 0 — Expectations and Recap

AmaliTech · Python Chapter

2025

Contents

Welcome

The Two Deployment Models

AWS Services We Will Use

Concepts You Should Remember

What to Prepare

Quick Glossary

Welcome

This is a two-session series on deploying Python applications to **Amazon Web Services (AWS)**. The hands-on demos are in **Session 1** and **Session 2**. This Session 0 doc is a 10-minute recap of the concepts you will meet — intentionally short.

The series

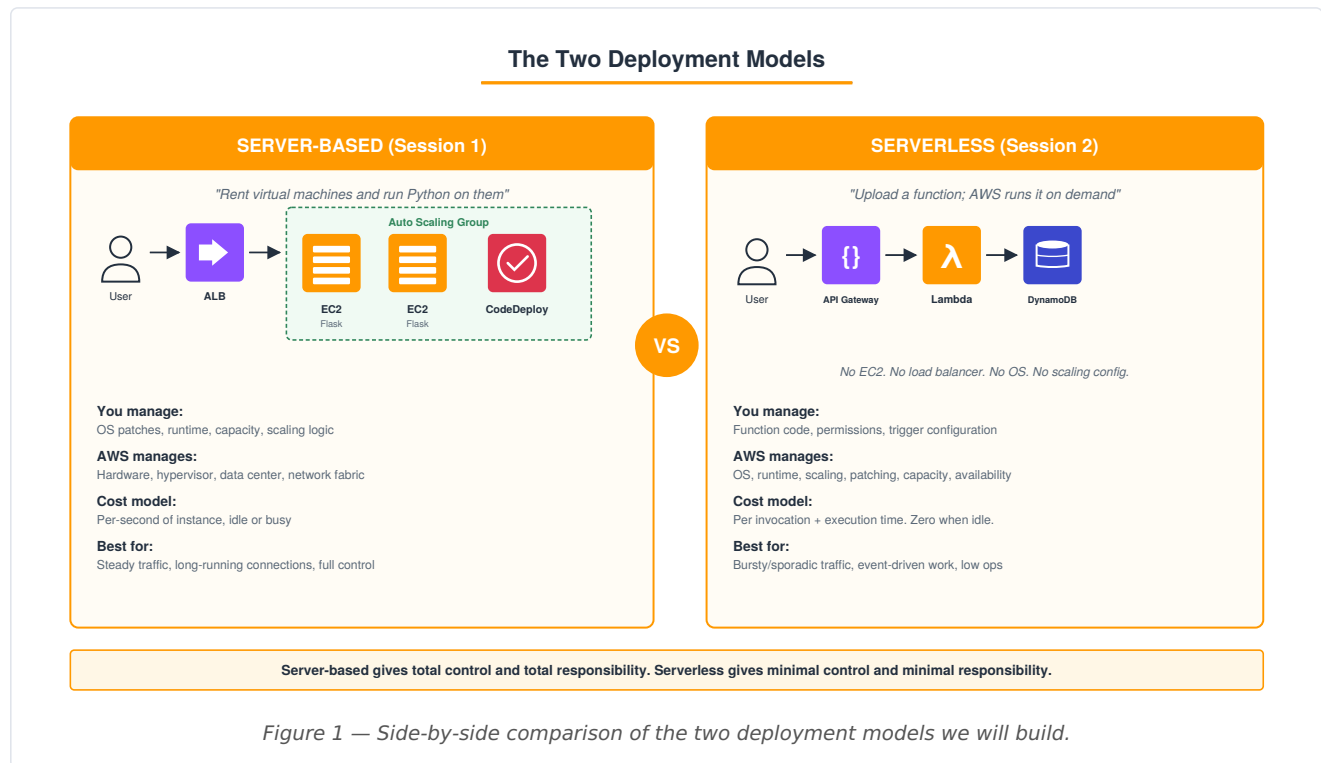
- **Session 1 — Server-Based Deployment.** A Python Flask app on EC2, behind a Load Balancer, with Auto Scaling, deployed automatically by AWS CodeDeploy.
- **Session 2 — Serverless Deployment.** The same kind of work done with Lambda, API Gateway, S3 and DynamoDB — no servers to manage.

Each session has an optional **Section A** showing CI/CD with GitHub Actions: a push to `main` triggers an automatic deploy.

Who this is for: Python developers with AWS Cloud Practitioner-level fundamentals — you know what EC2, S3, IAM are conceptually. You are comfortable on the Linux command line. We assume **Ubuntu Linux** on your laptop.

The Two Deployment Models

This is the most important framing for both sessions.



Server-based (Session 1): you rent virtual machines (EC2), install Python on them, run your code as a service. You manage the OS, the runtime, capacity, scaling. You pay per second of instance, idle or busy.

Serverless (Session 2): you upload Python code. AWS runs it on demand. You manage the code, permissions, and triggers — that's it. You pay per invocation. When nothing runs, you pay zero.

Choose server-based when: traffic is steady, you need full OS control, requests can run longer than 15 minutes, or you need long-lived connections.

Choose serverless when: traffic is bursty or unknown, the work is event-driven (file arrived, message received), you want zero operational overhead, or you want to pay nothing when idle.

Most real systems use both. A common pattern: serverless for background jobs and event handlers, server-based for steady-traffic web tiers.

AWS Services We Will Use

Session 1 (server-based)

- **EC2** — virtual machines that run your code
- **VPC + Security Groups** — network and firewall
- **Application Load Balancer (ALB)** — distributes traffic across instances
- **Auto Scaling Group (ASG)** — keeps a target number of instances running, replaces failed ones
- **S3** — stores the deployment zip
- **CodeDeploy** — rolls out new versions to instances with zero downtime
- **IAM** — controls who/what can perform which AWS actions

Session 2 (serverless)

- **Lambda** — runs Python functions on demand
- **API Gateway** — turns a Lambda into a public HTTPS endpoint
- **S3** — object storage; also a Lambda trigger when files are uploaded
- **DynamoDB** — fast NoSQL database, on-demand pricing
- **CloudWatch Logs** — captures every Lambda `print` and exception
- **IAM** — same role as in Session 1

Concepts You Should Remember

Region and Availability Zone. A **Region** is a geographic area (we use `eu-west-1` — Ireland). Each Region has multiple isolated **Availability Zones (AZs)**. Resources are Region-scoped — an EC2 instance in `eu-west-1` does not exist anywhere else. For high availability, run across at least 2 AZs.

IAM in one paragraph. Every action in AWS is checked against an IAM policy. A **role** is an identity that AWS services (EC2, Lambda) assume to do work. A role has policies attached that say what it can do. **Least privilege** means the role can do exactly what it needs, no more. We attach roles directly to instances and Lambda functions — no access keys floating around.

The deployment lifecycle. Every deployment has the same six steps: **source** (the code lives somewhere), **build** (we package it), **test** (run it locally), **distribute** (upload to AWS), **run** (start the new version), **verify** (health check). Session 1 does this manually, then automates it with CodeDeploy. Session 2 collapses several steps into one upload.

Logs are how you debug AWS. When something does not work, the answer is in **CloudWatch Logs**. Every Lambda print, every unicorn error, every system log lands there.

What to Prepare

- An AWS account where you can create resources (the AWS Free Tier is enough)
- A web browser for the AWS Management Console
- An Ubuntu Linux terminal for local development and `curl` tests
- Python 3.12 installed locally
- A GitHub account
- AWS CLI v2 installed locally (only needed for the optional CI/CD section)

Set the AWS Region to `eu-west-1` in the top-right of the Console before doing anything. Tag every resource with `Project = python-chapter-demo` so cleanup at the end is trivial.

Quick Glossary

Term	Meaning
AMI	Amazon Machine Image — the OS image an EC2 instance boots from
ALB	Application Load Balancer
ASG	Auto Scaling Group
AZ	Availability Zone
boto3	AWS's official Python SDK
CIDR	IP address range, e.g. <code>10.0.0.0/16</code>
EC2	Elastic Compute Cloud — AWS's virtual machines
IAM	Identity and Access Management
OIDC	OpenID Connect — used to give GitHub Actions short-lived AWS credentials
Region	A geographic group of AWS data centers, e.g. <code>eu-west-1</code>
SG	Security Group — virtual firewall around EC2 instances and load balancers
VPC	Virtual Private Cloud — your own isolated network in AWS
WSGI	Web Server Gateway Interface — Python's standard for web servers; <code>gunicorn</code> is a WSGI server

That's everything. **Session 1** and **Session 2** take it from here, hands-on.